Sistema de monitoreo y control distribuido

Propuesta de diseño y arquitectura (Fase 1)

ST0255 Telemática: Internet, Arquitectura y Protocolos

Emmanuel Hernández

Profesor: Jaime A. Vergara

Universidad EAFIT, semestre 2026-2

4 de octubre de 2026

Contenido

1. Resumen
2. Descripción general del problema
3. Alcance y supuestos
4. Arquitectura propuesta
5. Entidades participantes
6. Flujos y tipos de mensajes
7. Sintaxis preliminar
8. Reglas básicas de comunicación
9. Máquinas de estado
10. Análisis preliminar sobre el uso de TCP y UDP
11. Excepciones y errores
12. Autenticación y perfiles
13. Resolución de nombres (DNS)
14. Concurrencia prevista
15. Logging
16. Ejemplos completos
17. Requisitos que este diseño deja fijados para las Fases 2 y 3
18. Decisiones descartadas
19. Conclusiones
20. Referencias

Correspondencia con lo que pide la Fase 1

El enunciado pide que la propuesta inicial incluya ocho puntos y anuncia una exposición corta (Universidad EAFIT, 2026c). Esta tabla indica dónde está cada uno.

Punto literal del enunciado (Fase 1)	Dónde se responde
descripción general del problema	Sección 2
arquitectura propuesta	Sección 4 y docs/diagramas/despliegue.mmd
entidades participantes	Sección 5
tipos de mensajes	Sección 6 y especificación, sección 5
sintaxis preliminar	Sección 7 y especificación, sección 4
reglas básicas de comunicación	Secciones 8 y 11
máquinas de estado	Sección 9 y los cuatro diagramas fsm-*.mmd
análisis preliminar sobre el uso de TCP y UDP	Sección 10
exposición corta de unos cinco minutos	docs/entrega1/exposicion.md

La especificación completa del protocolo está en docs/protocolo/MCD-1.0.md. Esta propuesta explica las decisiones; la especificación las fija con el detalle necesario para implementarlas y toma como referencia la organización de una especificación clásica de transporte (Postel, 1981). Los dos documentos usan los mismos nombres, valores y límites. La correspondencia completa con todos los requisitos del enunciado está en docs/entrega1/trazabilidad.md.

A lo largo del documento separo tres cosas: lo que exige el enunciado (lo cito como "el enunciado pide" o por su número de requisito), lo que decidí yo para cumplirlo ("decisión") y lo que queda para las fases siguientes ("Fase 2" o "Fase 3").

---

1. Resumen

El proyecto es un sistema donde varios nodos le reportan su estado a un servidor central y unos clientes de administración consultan ese estado. Para coordinar esa comunicación diseñé un protocolo de aplicación propio, MCD/1.0 (Monitoreo y Control Distribuido), que se apoya en los dos tipos de sockets Berkeley.

La idea central del diseño es que no todos los mensajes tienen las mismas necesidades. La telemetría periódica es pequeña, frecuente y reemplazable: si se pierde una muestra, la siguiente llega pocos segundos después. Por eso va por UDP, sin confirmación, con un número de secuencia que permite detectar pérdidas, duplicados y desorden. El registro de los nodos, los eventos críticos, la autenticación y las consultas no toleran pérdidas y crean o leen estado, así que van por TCP. Los eventos llevan además una confirmación de aplicación y un identificador que evita guardarlos dos veces (Eddy, 2022; Postel, 1980).

El servidor está en C, atiende con un hilo por conexión TCP, un hilo para UDP y un hilo de vigilancia de temporizadores (watchdog). Los nombres se resuelven con un DNS propio (BIND9, zona monitoreo.lab) y la autenticación de usuarios y nodos se hace contra OpenLDAP, con dos perfiles: admin y operador (Universidad EAFIT, 2026a).

2. Descripción general del problema

Una organización tiene equipos repartidos (sensores, equipos de red, dispositivos IoT) que necesita supervisar de forma remota. Cada equipo, que aquí llamo nodo, debe informar a un servidor central dos tipos de información:

- Su estado periódico: uso de CPU, temperatura, nivel de batería, estado operativo y tiempo encendido. El enunciado aclara que perder alguno de estos mensajes de vez en cuando es tolerable, siempre que el dato se siga actualizando.
- Eventos críticos: fallas, superación de umbrales, pérdida de conectividad, alarmas o cambios importantes de estado. Aquí perder un mensaje sí tiene consecuencias.

El servidor central tiene que juntar lo que llega de varios nodos, mantener una foto actualizada de la infraestructura (qué nodos existen, cuáles están reportando y cuál fue su último estado), guardar un historial reciente y responder a los clientes de administración que preguntan por todo esto. Esos clientes no hablan con los nodos; todo pasa por el servidor.

El problema de fondo, y lo que el curso quiere evaluar, es diseñar el protocolo que coordina esta comunicación y decidir, para cada tipo de mensaje, qué servicio de transporte le conviene. Además el sistema tiene que integrarse con servicios reales de Internet: resolución de nombres en lugar de direcciones fijas y un servicio de autenticación centralizado en lugar de una lista de usuarios dentro del programa.

3. Alcance y supuestos

3.1 Qué cubre esta entrega

Esta es la Fase 1: diseño y arquitectura. Entrego la propuesta, la especificación del protocolo, las máquinas de estado, los diagramas de secuencia y de despliegue, la matriz de trazabilidad y el guion de la exposición. Todavía no hay código. Cuando en este documento digo que el servidor "hace" algo, describo cómo lo voy a implementar, no algo que ya funcione.

3.2 Supuestos

Supuesto	Por qué lo asumo
Todo corre en máquinas virtuales Ubuntu Server 24.04 en VirtualBox, en una red host-only.	Es el entorno de pruebas del curso. AWS es opcional en el enunciado y lo dejo como posibilidad para el final.
Hay dos nodos simulados en Python.	El enunciado pide al menos 2 nodos. Simular las métricas permite provocar eventos y fallas a voluntad para las pruebas.
Hay un solo cliente de administración, pero el diseño admite varios a la vez.	El enunciado pide "uno o varios" clientes y que el servidor sea concurrente.
Los relojes de las máquinas no necesitan estar sincronizados.	El servidor decide si un nodo dejó de reportar con su propio reloj. El ts que envía el nodo solo se muestra.
La red del laboratorio no es hostil.	Las credenciales viajan sin cifrar. Lo declaro como limitación en la sección 12.
El estado del servidor vive en memoria.	El enunciado pide estado actual e historia reciente (al menos 5 datos), no persistencia. Si el servidor se reinicia, los nodos se vuelven a registrar solos.

3.3 Fuera del alcance

- Cifrado (TLS o LDAPS).
- Persistencia en disco del historial.
- Que los clientes envíen órdenes a los nodos. El nombre del proyecto dice "control", pero el enunciado no pide comandos hacia los nodos y además prohíbe la comunicación directa entre nodos y clientes. Si hiciera falta, se agregaría un tipo de mensaje que el servidor reenvíe por la conexión TCP del nodo, sin cambiar el formato del protocolo.

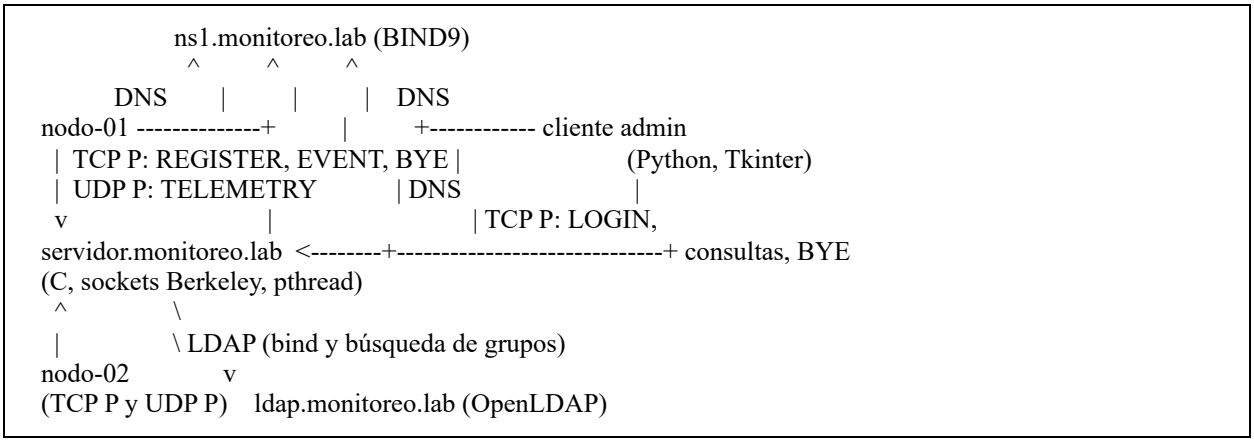
3.4 Condiciones de la entrega

Esta entrega tiene un solo autor: Emmanuel Hernández. El plazo de la Fase 1 fue extendido hasta el 4 de octubre de 2026.

4. Arquitectura propuesta

4.1 Componentes

El diagrama completo está en docs/diagramas/despliegue.mmd.



Máquina lógica	Componente	Qué escucha
servidor.monitoreo.lab	Servidor central en C	Puerto P en TCP y en UDP (por ejemplo, 5050)
ns1.monitoreo.lab	BIND9 con la zona monitoreo.lab	53 en UDP y TCP
ldap.monitoreo.lab	OpenLDAP (slapd) con base dc=monitoreo,dc=lab	389 TCP
Máquinas de los nodos	nodo-01 y nodo-02 en Python	Nada; solo inician conexiones
Máquina del administrador	Cliente Python con Tkinter	Nada; solo inicia conexiones

Son máquinas lógicas. En la práctica el DNS y el LDAP pueden compartir una misma VM, y los dos nodos también, sin que cambie nada del diseño, porque todos se encuentran por nombre.

4.2 Decisiones de arquitectura

- Modelo cliente-servidor estricto: el servidor es el único que escucha. Los nodos y los clientes siempre inician. Esto cumple el requisito 2 del enunciado (no hay comunicación directa entre nodos y clientes) y además funciona aunque los nodos estén detrás de NAT.
- Un solo número de puerto P para TCP y para UDP. El enunciado pide que el servidor reciba un único parámetro "puerto". TCP y UDP tienen espacios de puertos separados (un

socket se identifica por protocolo, IP y puerto de cada extremo), así que abrir los dos en el mismo número no genera conflicto.

- Un mismo puerto TCP para nodos y para clientes. El servidor distingue de qué se trata por el primer mensaje: REGISTER o LOGIN. Así no necesito un segundo puerto que el enunciado no contempla.
- Conexión TCP persistente por nodo. El nodo la abre al registrarse y la mantiene; por ahí van los eventos. Evita el costo de abrir una conexión por evento y le permite al servidor saber cuándo un nodo se desconectó.

5. Entidades participantes

Entidad	Lenguaje	Papel	Habla con	Transporte
Servidor central	C (el enunciado lo exige)	Registra nodos, recibe telemetría y eventos, mantiene el estado y el historial, autentica, responde consultas y escribe el log	Nodos, clientes, LDAP, DNS	TCP y UDP en P; LDAP sobre TCP 389; DNS
Nodo	Python	Se registra, envía telemetría cada period segundos y notifica eventos	Servidor, DNS	TCP y UDP hacia P; DNS
Cliente de administración	Python con Tkinter	Autentica a una persona, muestra el estado actual de los nodos, las últimas muestras de cada uno y, para el perfil admin, los eventos	Servidor, DNS	TCP hacia P; DNS
Servidor DNS	BIND9	Resuelve los nombres de la zona monitoreo.lab	Todos	UDP/TCP 53
Directorio LDAP	OpenLDAP	Guarda usuarios (ou=people), nodos (ou=devices) y grupos de perfil (ou=groups)	Solo el servidor	TCP 389

Los usuarios humanos tienen uno de dos perfiles:

- operador: ve la lista de nodos, el estado de cada uno y su historial.
- admin: todo lo anterior y además la lista de eventos, incluidos los que genera el propio servidor cuando un nodo deja de reportar.

6. Flujos y tipos de mensajes

6.1 Operaciones mínimas del enunciado

El requisito 7 pide mensajes para siete operaciones. Así las cubre MCD/1.0:

Operación del requisito 7	Mensajes	Transporte
a) registro de un nodo	REGISTER	TCP
b) envío de información periódica de estado	TELEMETRY	UDP
c) envío de un evento	EVENT	TCP
d) confirmación o respuesta cuando sea necesaria	REGISTER_OK, ACK, LOGIN_OK, BYE	TCP
e) consulta de información	LIST, STATUS, HISTORY, EVENTS (previo LOGIN)	TCP
f) respuesta a una consulta	RESPONSE seguido de líneas ITEM	TCP
g) reporte de errores	ERROR	TCP; por UDP solo el 404

En total son 15 tipos de mensaje. El detalle de campos de cada uno está en la especificación, sección 5.

6.2 Flujos

1. Registro (secuencia-registro.mmd): el nodo resuelve el nombre del servidor, abre TCP y envía REGISTER. El servidor valida el node_id y el secret contra LDAP y contesta REGISTER_OK con un token de sesión.
2. Telemetría (secuencia-telemetria.mmd): cada period segundos el nodo envía un datagrama TELEMETRY con el token de sesión y un número de secuencia. No hay respuesta.
3. Evento (secuencia-evento.mmd): el nodo envía EVENT por su conexión TCP y espera ACK. Si no llega, se reconecta y lo reenvía con el mismo identificador; el servidor no lo duplica.
4. Consulta (secuencia-consulta.mmd): el cliente envía LOGIN, el servidor verifica en LDAP y responde con el perfil. Después el cliente consulta cada 5 segundos.
5. Errores (secuencia-errores.mmd) y servicios caídos (secuencia-servicios-caidos.mmd): cómo responde el servidor ante mensajes inválidos, temporizadores vencidos y fallas de DNS o LDAP.

6.3 Por qué RESPONSE más ITEM

Una consulta puede devolver varios registros (10 muestras de historial, 20 eventos). Meterlos todos en un campo obligaría a usar separadores como comas o punto y coma, que dejé por fuera de los caracteres permitidos, y además el historial completo no cabría en 512 bytes. Por eso la respuesta es una línea RESPONSE que dice cuántas líneas ITEM siguen. El cliente sabe exactamente cuántas debe leer y cada línea sigue siendo un mensaje normal, con la misma gramática y el mismo límite de tamaño.

7. Sintaxis preliminar

Cada mensaje es una línea de texto ASCII:

```
MCD/1.0|TIPO|clave=valor|clave=valor|...\n
```

Reglas principales (la gramática ABNF completa está en la especificación, sección 4.2, y sigue la notación definida por Crocker y Overell (2008), con la extensión de Kyzivat (2014)):

- Empieza con la versión `MCD/1.0` y luego el tipo en mayúsculas.
- Los campos son pares `clave=valor` separados por `|`. Las claves van en minúscula.
- Los valores solo pueden tener letras ASCII, dígitos, punto, guion, guion bajo y dos puntos. No se permiten `|`, `=`, espacios ni caracteres fuera de ASCII. Si algún campo necesitara texto libre, se usa un código enumerado.
- El mensaje termina en un único `\n` y mide como máximo 512 bytes, contando el `\n`.
- Hasta 16 campos por mensaje, sin claves repetidas, de 1 a 64 caracteres por valor.

Ejemplos:

```
MCD/1.0|REGISTER|req=1|node_id=nodo-01|secret=clave-de-ejemplo|kind=SENSOR|period=5
MCD/1.0|TELEMETRY|node_id=nodo-01|session=4f9c2a7d1e0b8c63|seq=41|ts=1791126131|cpu=37.5|temp=48.1|bat=91|opstate=OK|uptime=3731
MCD/1.0|ERROR|req=4|code=422|reason=VALOR|field=n
```

Por qué texto y por qué así:

- Texto legible: puedo probar el servidor con `nc` antes de tener los clientes, y en Wireshark se lee cada mensaje sin decodificador. Es la misma idea de HTTP y SMTP que vimos en clase.
- Delimitado por `\n`: TCP entrega un flujo de bytes, no mensajes. Necesito una marca de fin para separar los mensajes cuando un `recv()` trae medio mensaje o varios juntos. Un salto de línea es la marca más simple de depurar.
- 512 bytes de máximo: el mensaje más grande que puede existir (un `ITEM` de estado con todos los campos al máximo) mide como máximo 245 bytes, así que sobra margen.

Además un datagrama de 512 bytes más los encabezados UDP e IP (540 bytes en total) nunca se fragmenta en Ethernet.

- Caracteres restringidos: evitan que un valor rompa la estructura del mensaje, que inyecte algo en una consulta LDAP o que dañe una línea del log. También simplifican el parser en C.

Campos que se repiten en varios mensajes:

Campo	Para qué sirve
req	Número de petición en una conexión TCP. La respuesta lo copia, así el que pregunta sabe a qué le están respondiendo
node_id	Nombre del nodo, igual al registrado en LDAP
session	Token de 16 caracteres hexadecimales que entrega el servidor al registrar. Va en cada telemetría
seq	Número de secuencia de la telemetría, de a 1 por sesión
msg_id	Identidad de un evento, de la forma <arranque>-<contador>
ts	Hora en segundos desde 1970 (epoch) según quien genera el dato

8. Reglas básicas de comunicación

Las reglas completas, con algoritmos, están en la especificación, sección 7. Las esenciales son:

1. El servidor es el único que escucha; nodos y clientes inician siempre.
2. La primera petición en una conexión TCP debe ser REGISTER (nodo) o LOGIN (cliente). Antes de eso, una consulta o un evento reciben 401 SIN_SESION. Una conexión no puede cambiar de identidad.
3. Ningún nodo envía telemetría ni eventos antes de recibir REGISTER_OK. Ningún cliente consulta antes de LOGIN_OK.
4. Toda petición TCP lleva req y toda respuesta lo copia. Nodos y clientes envían una petición a la vez y esperan la respuesta antes de enviar otra.
5. La telemetría solo va por UDP y no se responde. El servidor la acepta solo si session es la vigente. Un seq repetido se descarta como duplicado; uno menor al último se descarta como atrasado; un salto suma al contador de perdidos.
6. Cada evento se confirma con ACK solo después de guardarlo. El servidor recuerda los últimos 64 msg_id de cada nodo y, si llega uno repetido, responde ACK con dup=1 sin guardarlo otra vez.
7. Un nodo pasa a SIN_REPORTE si pasan más de 3 periodos sin telemetría válida, y vuelve a ACTIVO con la siguiente que llegue.
8. Si el servidor responde 404 por UDP, el nodo se registra de nuevo. Si el servidor se reinicia, así se recupera todo sin intervención.
9. Cualquier error se reporta con ERROR, con un código numérico y un motivo enumerado. Los errores 400 o 422 no cierran la conexión, salvo que haya 5 seguidos.
10. Ninguna falla de DNS, LDAP o red termina un proceso. Todo reintento usa la misma espera creciente: 1, 2, 4, 8, 16 y 30 segundos.
11. El cierre ordenado es con BYE, que el servidor contesta con otro BYE antes de cerrar.

9. Máquinas de estado

Hay cuatro máquinas. Cada una tiene su diagrama en docs/diagramas/ y su tabla de transiciones completa en la sección 9 de la especificación. Aquí resumo qué controla cada una:

- **Nodo (fsm-nodo.mmd)**: pasa por resolución DNS, conexión, registro y operación. Una falla lleva a ESPERA_REINTENTO; un evento espera su ACK sin detener la telemetría periódica.
- **Nodo visto por el servidor (fsm-servidor-nodo.mmd)**: empieza como DESCONOCIDO, queda ACTIVO después de registrarse y pasa a SIN_REPORTE si pasan más de tres periodos sin telemetría. Un cierre TCP lo deja DESCONECTADO.
- **Sesión administrativa (fsm-sesion-admin.mmd)**: exige LOGIN, limita los intentos y solo permite consultas después de autenticar el perfil en LDAP.
- **Cliente de administración (fsm-cliente.mmd)**: obtiene credenciales, resuelve el servidor, inicia sesión, consulta y vuelve a conectarse si falla la red. Un 401 o 403 lo devuelve al formulario.

Los diagramas muestran las transiciones; la especificación define de manera normativa el evento, la acción y el siguiente estado de cada una.

10. Análisis preliminar sobre el uso de TCP y UDP

El enunciado pide analizar cada tipo de mensaje según nueve criterios y no simplemente declarar un protocolo "mejor" (Universidad EAFIT, 2026b). Agrupo los mensajes en cuatro flujos, porque los mensajes de un mismo flujo comparten necesidades (por ejemplo, RESPONSE e ITEM viajan siempre como respuesta a una consulta).

10.1 Matriz por criterio

Criterio	Registro (REGISTER, REGISTER_OK)	Telemetría (TELEMETRY)	Eventos (EVENT, ACK)	Autenticación y consultas (LOGIN, LIST, STATUS, HISTORY, EVENTS, RESPONSE, ITEM, BYE)
Frecuencia de transmisión	Una vez por arranque o reconexión	Cada period segundos por nodo (5 s por defecto)	Esporádica, cuando algo pasa	Un LOGIN por sesión; luego unas pocas consultas cada 5 s
Tamaño de los mensajes	84 y 76 bytes en el ejemplo; máximo 166	129 bytes en el ejemplo; máximo 182	113 y 60 bytes en el ejemplo; máximo 188	Consultas de 19 a 76 bytes; respuestas de 1 línea RESPONSE más hasta 32 líneas ITEM (un LIST con la tabla llena), cada una de hasta 245 bytes
Criticidad de la información	Alta: sin registro no hay sesión ni telemetría válida	Baja por muestra: la siguiente reemplaza a la anterior	Alta: una falla o una alarma no se puede perder	Alta: credenciales y datos que la persona va a mirar
Tolerancia a pérdidas	Ninguna	Alta, siempre que la pérdida sea ocasional	Ninguna	Ninguna
Necesidad de orden	Sí: el registro debe llegar antes que cualquier evento	No: una muestra atrasada se descarta	Sí entre eventos de un mismo nodo	Sí: RESPONSE debe preceder a sus ITEM y estos ir en orden
Necesidad de confiabilidad	Sí	No	Sí, y además confirmación de que se procesó	Sí
Necesidad de mantener una conexión	Sí: la conexión identifica al nodo y detecta su desconexión	No	Sí, se reutiliza la del registro	Sí: la sesión autenticada es la conexión
Comportamiento ante retardos	Se puede esperar unos segundos (T_resp de 5 s)	Un dato retrasado pierde valor; mejor que no bloquee a los siguientes	Se tolera un retardo de segundos; lo que importa es que llegue una vez	La persona tolera algunos segundos; después se reconecta
Comportamiento ante pérdida de mensajes	TCP retransmite; si la conexión se cae, el nodo se registra de nuevo	Se cuenta como perdida y se sigue; nadie la retransmite	TCP retransmite; si se cae la conexión sin ACK, el nodo reenvía con el mismo msg_id	TCP retransmite; si se cae la conexión, el cliente reconecta y repite la consulta

Criterio	Registro (REGISTER, REGISTER_OK)	Telemetría (TELEMETRY)	Eventos (EVENT, ACK)	Autenticación y consultas (LOGIN, LIST, STATUS, HISTORY, EVENTS, RESPONSE, ITEM, BYE)
Transporte elegido	TCP	UDP	TCP con ACK de aplicación	TCP

10.2 Telemetría por UDP

La telemetría cumple casi palabra por palabra lo que el enunciado describe como información donde "la pérdida ocasional puede ser tolerable, siempre que la información sea actualizada periódicamente". Con TCP, una muestra perdida se retransmitiría y, mientras tanto, las muestras siguientes quedarían detenidas detrás de ella esperando su turno. Eso es justo lo contrario de lo que quiero: el dato nuevo vale más que el viejo.

UDP además me da otras ventajas en este flujo:

- No hay conexión que mantener por cada nodo para la telemetría, ni saludo de tres vías.
- Cada muestra es independiente y cabe en un datagrama.
- La carga es baja: 129 bytes de mensaje más 28 de encabezados UDP e IPv4 cada 5 segundos son unos 31 bytes por segundo por nodo.

La limitación es que UDP no avisa nada: ni pérdidas, ni duplicados, ni desorden. De los mecanismos que el enunciado lista para UDP, implemento solo los que este flujo necesita:

Mecanismo del enunciado	¿Se usa en la telemetría?	Cómo
confirmación de recepción	No	Confirmar cada muestra duplicaría el tráfico para proteger datos que de todas formas se reemplazan
identificación de mensajes	Sí	node_id y session identifican el origen; seq identifica la muestra
temporizadores	Sí, en el servidor	El watchdog detecta más de 3 periodos sin datos y marca SIN_REPORTE
retransmisión	No	Una muestra retransmitida llegaría vieja
detección de duplicados	Sí	seq igual al último se descarta y se cuenta
control de mensajes fuera de orden	Sí	seq menor al último se descarta y se cuenta

10.3 Eventos por TCP y no por UDP con confirmación

Los eventos necesitan llegar, llegar una sola vez y en orden. Si los mandara por UDP tendría que implementar confirmaciones, temporizadores, retransmisión, detección de duplicados y orden: prácticamente rehacer TCP, que el enunciado pide no hacer. TCP ya da confiabilidad y orden mientras la conexión está viva.

Lo que TCP no resuelve es un caso concreto: el servidor guardó el evento, pero la conexión se cayó antes de que el nodo recibiera la confirmación. El nodo no sabe si el evento llegó. Para ese caso agrego solo dos cosas en la aplicación:

- Un ACK que el servidor envía después de guardar el evento. Significa "procesado", no solo "recibido", que es lo único que TCP garantiza.
- Un msg_id estable para que el reenvío después de reconectar no se guarde dos veces.

El costo de usar TCP aquí es bajo: los eventos son esporádicos y la conexión ya está abierta por el registro.

10.4 Registro, autenticación y consultas por TCP

Estos flujos crean o leen estado y siguen un patrón de petición y respuesta. Necesitan las tres garantías de TCP (confiabilidad, orden y conexión). Además la conexión misma representa algo: la sesión autenticada de un cliente o la presencia de un nodo. Cuando recv() devuelve 0, el servidor sabe que el otro lado se fue. Con UDP tendría que inventar todo eso.

10.5 Resultado

La arquitectura combina TCP y UDP, que es una de las tres opciones que permite el enunciado: UDP solo para la telemetría y TCP para todo lo demás. Lo que agrego en la aplicación (secuencia en la telemetría, confirmación e identificador en los eventos) responde a un problema concreto de cada flujo.

11. Excepciones y errores

El requisito 11 pide control de excepciones y la Fase 3 enumera las situaciones que se van a evaluar. Esta tabla dice cómo se comporta el sistema en cada una. Los códigos completos están en la especificación, sección 6.

Situación (lista de la Fase 3 y requisito 11)	Comportamiento definido
Mensajes con formato incorrecto	400 (FORMATO, VERSION o TIPO). La conexión sigue; al quinto error seguido se cierra. Por UDP se descarta sin respuesta y se registra
Mensaje demasiado largo	413 TAMANO y cierre, porque ya no es seguro saber dónde empieza el siguiente mensaje
Parámetros inválidos	422 con el motivo (FALTA_CAMPO, CAMPO_DESCONOCIDO, CAMPO_REPETIDO o VALOR) y el campo culpable en field
Desconexiones	recv() devuelve 0 o -1: el servidor libera la conexión y, si era un nodo, lo marca DESCONECTADO. El nodo y el cliente reconectan con espera creciente
Fallas en la comunicación (conexión rechazada, sin ruta, send fallido)	Se registran y se reintenta con espera creciente. El servidor ignora SIGPIPE para no morir al escribir en un socket cerrado
Nodos desconocidos	Consulta sobre un nodo que no existe: 404 NODO. Telemetría de un nodo desconocido o con sesión vieja: 404 por UDP, como máximo uno cada 5 s por origen
Expiración de temporizadores	T_ident (60 s sin identificarse) y T_inactividad (5 min sin peticiones): 408 y cierre. T_ack en el nodo: reconexión y reenvío. T_resp en nodo y cliente: se trata como falla de red. Watchdog: SIN_REPORTE
Pérdida de mensajes	Telemetría: se cuenta y se sigue. Evento: reenvío tras T_ack
Duplicación de mensajes	Telemetría: seq repetido se descarta. Evento: msg_id repetido se confirma con dup=1 y no se guarda dos veces
Credenciales inválidas o perfil insuficiente	401 CREDENCIALES (3 intentos por conexión) y 403 ROL
Mensaje en un estado que no corresponde	401 SIN_SESION si falta identificarse; 409 ESTADO si ya se identificó; 409 TRANSPORTE si llega telemetría por TCP
Falla de DNS	Se registra, se reintenta con espera creciente y el proceso no termina (requisito 4)
Falla del LDAP	503 al que pidió, espera creciente antes de volver a contactar al LDAP, el servidor sigue atendiendo
Servidor lleno	503 CAPACIDAD al aceptar la conexión número 65 o al registrar el nodo número 33
Falla interna	500 INTERNO, se cierra solo esa conexión

Para que el comportamiento sea predecible, la especificación fija un orden de validación (sección 6.1): primero el tamaño, luego la gramática, la versión, el tipo, el canal, el estado de la conexión, los campos y al final la semántica. Así dos implementaciones distintas responden el mismo código ante el mismo mensaje.

12. Autenticación y perfiles

El requisito 6 pide autenticación y gestión de perfiles, y recomienda no basarla en archivos locales ni en una base de datos dentro de la aplicación. Decidí usar OpenLDAP, el servicio de directorio estudiado en clase (Sermersheim, 2006; Universidad EAFIT, 2026a).

Estructura del directorio:

```
dc=monitoreo,dc=lab
├── ou=people     uid=admin1, uid=operador1, ...   (usuarios humanos)
├── ou=devices    cn=nodo-01, cn=nodo-02           (nodos)
└── ou=groups     cn=admins, cn=operadores         (groupOfNames, atributo member)
```

Cómo se usa:

- Usuarios: el servidor hace un *bind* simple con `uid=<user>,ou=people,dc=monitoreo,dc=lab` y la clave recibida. Si el LDAP acepta, busca a qué grupo pertenece el DN. `cn=admins` da el perfil `admin`; `cn=operadores` da `operador`. Si no está en ninguno, se rechaza con 403.
- Nodos: el mismo mecanismo con `cn=<node_id>,ou=devices,dc=monitoreo,dc=lab`. Así un equipo que no esté dado de alta en el directorio no puede registrarse.
- El servidor no guarda contraseñas. Pregunta al LDAP cada vez.

Permisos:

Operación	admin	operador	Nodo
Ver lista, estado e historial	Sí	Sí	No
Ver eventos	Sí	No	No
Enviar eventos y telemetría	No	No	Sí

El operador vigila el estado del momento; el administrador además investiga incidentes, y para eso necesita la lista de eventos.

Limitaciones y punto abierto:

- Las credenciales viajan en claro, tanto en MCD/1.0 como en el *bind* LDAP. En una red de laboratorio lo acepto; en producción haría falta TLS.

- Para hablar con el LDAP desde C pienso usar libldap. Esa biblioteca consume un servicio estándar; los sockets del protocolo propio se construyen solo con la API Berkeley, como pide el enunciado. Lo voy a confirmar con el profesor antes de la Fase 2. Si no se acepta, la alternativa es validar con PAM (libpam-ldapd), que consulta el mismo directorio.
- Las claves reales no van al repositorio. Los ejemplos usan clave-de-ejemplo.

13. Resolución de nombres (DNS)

El requisito 4 prohíbe direcciones IP en el código y exige que una falla de resolución no termine el servicio. La resolución se apoya en el modelo DNS descrito por Mockapetris (1987a, 1987b).

- Zona monitoreo.lab en BIND9, con registros para ns1, servidor y ldap. Las máquinas apuntan su resolvedor a ns1 (en Ubuntu hay que tener en cuenta systemd-resolved, como vimos en el laboratorio de DNS).
- El nodo y el cliente reciben el nombre del servidor y el puerto por configuración o por argumentos, y resuelven con socket.getaddrinfo en cada intento de conexión.
- El servidor lee el nombre del LDAP de la variable de entorno MCD_LDAP_HOST (por defecto ldap.monitoreo.lab) y lo resuelve con getaddrinfo antes de cada verificación. Así puede responder 503 DNS o 503 LDAP según dónde estuvo la falla.
- Para escuchar, el servidor usa getaddrinfo con AI_PASSIVE y sin nombre de host, lo que equivale a todas sus interfaces sin escribir ninguna dirección.
- Si la resolución falla (EAI_AGAIN, EAI_NONAME en C; socket.gaierror en Python), se registra y se reintenta con la espera creciente de la regla 10 (sección 8). El proceso sigue vivo.

14. Concurrencia prevista

El requisito 3f y el 10 piden atender varios clientes a la vez, y el enunciado permite hilos.

Decisión: un hilo por conexión TCP.

Hilo	Cuántos	Qué hace
Principal	1	Crea los sockets, hace bind y listen, acepta conexiones y lanza un hilo por cada una
Conexión TCP	Hasta 64	Lee con su propio buffer de encuadre, valida, responde y vigila sus temporizadores
Receptor UDP	1	recvfrom() en ciclo; valida y aplica cada telemetría
Watchdog	1	Cada segundo revisa qué nodos llevan más de 3 periodos sin reportar

Datos compartidos y cómo se protegen:

- Un mutex para la tabla de nodos, los historiales y la lista de eventos.
- Otro mutex para el log, para que las líneas de dos hilos no se mezclen.
- Orden fijo: si un hilo necesita los dos, toma primero el de la tabla y después el del log. Nunca al revés, así no hay interbloqueo.
- Las operaciones que pueden demorarse (consultas al LDAP, al DNS y los send()) se hacen sin el mutex de la tabla. Para responder una consulta, el hilo copia los datos con el mutex tomado, lo suelta y luego envía.

Por qué hilos y no procesos ni select():

- Con fork() cada proceso tendría su propia copia de la tabla y necesitaría memoria compartida o algún otro mecanismo para que el hilo UDP actualice lo que leen las conexiones TCP. Con hilos el estado se comparte directamente.
- select() o poll() en un solo hilo evitaría los mutex, pero una verificación LDAP lenta bloquearía a todos los demás clientes. Con un hilo por conexión, solo espera quien está autenticándose.
- La cantidad de conexiones esperada es pequeña (decenas), así que el costo de un hilo por conexión no es un problema. El límite de 64 evita agotar recursos.

15. Logging

El enunciado pide que el servidor muestre en consola las peticiones y respuestas, que también las guarde en un archivo y que registre la IP y el puerto de origen de cada una. El servidor se ejecutará así:

```
./servidor 5050 servidor.log
```

Formato de cada línea: fecha UTC, canal, dirección del otro extremo, sentido y mensaje.

```
2026-10-04T15:02:06Z TCP 192.168.56.21:51522 <- MCD/1.0|REGISTER|req=1|node_id=nodo-01|secret=***|kind=SENSOR|period=5
2026-10-04T15:02:06Z TCP 192.168.56.21:51522 -> MCD/1.0|REGISTER_OK|req=1|node_id=nodo-01|session=4f9c2a7d1e0b8c63|period=5
2026-10-04T15:02:11Z UDP 192.168.56.21:40311 <- MCD/1.0|TELEMETRY|node_id=nodo-01|session=4f9c2a7d1e0b8c63|seq=1|ts=1791126131|cpu=37.5|temp=48.1|bat=91|opstate=OK|uptime=3731
2026-10-04T15:02:17Z SYS - -- nodo-02 estado=SIN_REPORTE anterior=ACTIVO msg_id=srv-3
```

- <- es recibido, -> es enviado y -- es una nota interna (descartes, cambios de estado, fallas de DNS o LDAP).
- Las claves (secret, password) se escriben como ***.
- Los bytes no imprimibles de un mensaje malformado se escriben como \xNN, para que el log siga siendo legible.
- El archivo se abre en modo de agregar y se hace fflush después de cada línea, para no perder registros si el proceso se interrumpe.

Las direcciones de este ejemplo son las que vería en la red host-only de VirtualBox; no aparecen en ningún código.

16. Ejemplos completos

16.1 Un nodo se registra, reporta y notifica un evento

```
N->S TCP  MCD/1.0|REGISTER|req=1|node_id=nodo-01|secret=clave-de-ejemplo|kind=SENSOR|period=5
S->N TCP  MCD/1.0|REGISTER_OK|req=1|node_id=nodo-01|session=4f9c2a7d1e0b8c63|period=5
N->S UDP  MCD/1.0|TELEMETRY|node_id=nodo-
01|session=4f9c2a7d1e0b8c63|seq=1|ts=1791126131|cpu=37.5|temp=48.1|bat=91|opstate=OK|uptime=3731
N->S UDP  MCD/1.0|TELEMETRY|node_id=nodo-
01|session=4f9c2a7d1e0b8c63|seq=2|ts=1791126136|cpu=41.0|temp=49.0|bat=91|opstate=OK|uptime=3736
N->S TCP  MCD/1.0|EVENT|req=2|node_id=nodo-01|msg_id=1791122400-
1|ts=1791126138|level=CRITICO|event=UMBRAL_TEMP|value=85.2
S->N TCP  MCD/1.0|ACK|req=2|node_id=nodo-01|msg_id=1791122400-1|dup=0
```

El msg_id 1791122400-1 indica que el proceso del nodo arrancó en el segundo epoch 1791122400 (4 de octubre de 2026, 14:00 UTC) y que este es su primer evento.

Los demás intercambios completos (login, consultas, respuestas de varias líneas y casos de error) están en la sección 14 de docs/protocolo/MCD-1.0.md. Allí también se muestran mensajes que deben rechazarse. Todos los ejemplos respetan la gramática y el límite de 512 bytes.

17. Requisitos que este diseño deja fijados para las Fases 2 y 3

Varias exigencias del enunciado se evalúan en fases posteriores pero condicionan el diseño desde ya. Si no quedaran decididas ahora, habría que rehacer el protocolo después.

17.1 Fase 2: comunicación básica

Lo que la Fase 2 debe demostrar	Cómo lo prepara este diseño
creación y configuración de los sockets	getaddrinfo con AI_PASSIVE, SO_REUSEADDR, un socket TCP y uno UDP en el mismo puerto (Universidad EAFIT, 2026d; especificación, sección 10)
establecimiento de la comunicación	Identificación por primer mensaje (REGISTER o LOGIN)
envío de mensajes	Envío completo con reintento ante send() parcial (4.3 de la especificación)
recepción de mensajes	Buffer por conexión y separación por \n; un mensaje por datagrama
interpretación de mensajes	Gramática ABNF y tabla de campos con rangos
construcción de respuestas	REGISTER_OK, ACK, LOGIN_OK, RESPONSE e ITEM, ERROR
implementación de las reglas básicas del protocolo	Reglas de la sección 8 y procedimientos de la sección 7 de la especificación

Orden de implementación previsto: servidor TCP con argumentos y log; parser y encuadre; REGISTER y LOGIN; telemetría UDP; nodo; cliente; integración con DNS y LDAP.

17.2 Fase 3: concurrencia, resiliencia y pruebas

Lo que pide la Fase 3	Dónde queda definido
atender múltiples clientes o nodos de manera concurrente	Sección 14
mensajes con formato incorrecto, parámetros inválidos	Sección 11 y especificación, sección 6
desconexiones y fallas en la comunicación	Secciones 9 y 11
nodos desconocidos	404 NODO y SESION
expiración de temporizadores	Especificación, sección 8
pérdida y duplicación de mensajes	Secciones 10.2, 10.3 y 11

17.3 Requisitos generales que el diseño ya respeta

- Servidor solo en C, con la API de sockets Berkeley para el protocolo.
- Ejecución con ./servidor puerto archivoDeLogs; compilación con gcc y Makefile.
- Log en consola y archivo con IP y puerto de origen.
- Ninguna IP en el código; resolución por DNS con manejo de fallas.

- Al menos 2 nodos y un cliente con estado actual y al menos 5 datos históricos (el servidor guarda 10 por nodo y el cliente pide 5 por defecto).
- Autenticación y perfiles en un servicio externo (LDAP).

18. Decisiones descartadas

Estas son las alternativas que más pesaron en el diseño. Otras ya quedan justificadas donde se tomó la decisión: confirmar cada telemetría (sección 10.2), usar puertos distintos para nodos y clientes (sección 4.2), meter listas dentro de un campo (sección 6.3) y atender con un proceso por conexión (sección 14).

Alternativa	Por qué la descarté
Todo por TCP	Más simple, pero la telemetría sufriría bloqueos por retransmisión de datos que ya no sirven, y el análisis TCP/UDP quedaría solo en teoría
Todo por UDP	Obligaría a reimplementar confiabilidad, orden y sesión para registro, eventos y consultas; el enunciado pide no reimplementar TCP
Eventos por UDP con ACK, temporizador y retransmisión	Daría más control sobre la latencia, pero son cinco mecanismos más y más casos de error para un flujo esporádico que TCP ya resuelve
Protocolo binario con encabezado fijo	Mensajes más pequeños, pero hay que cuidar el orden de bytes (htons, ntohl), no se puede probar con nc y en Wireshark hace falta un decodificador. Con 129 bytes por telemetría el tamaño no es un problema
Usuarios en un archivo o en una base de datos propia	El enunciado lo desaconseja; LDAP es el servicio de directorio visto en clase
Un solo hilo con select()	Evitaría los mutex, pero una verificación LDAP lenta bloquearía a todos los demás clientes
msg_id como contador simple desde 1	Un nodo que se reinicia repetiría identificadores y el servidor descartaría eventos nuevos como duplicados. Por eso se antepone la hora de arranque
Marcar SIN_REPORTE tras un solo periodo sin datos	Una pérdida UDP aislada es normal y generaría falsas alarmas

19. Conclusiones

El punto de partida del diseño fue que el enunciado describe dos tipos de información con necesidades opuestas. Por eso la telemetría, donde importa más tener el dato reciente que recibir cada muestra, va por UDP, y todo lo que no se puede perder va por TCP. Encima del transporte agregué solo lo que cada flujo necesita: un número de secuencia en la telemetría y, en los eventos, una confirmación con un identificador estable.

El formato de texto, con un mensaje por línea, me permite probar el servidor con `nc` y comprobar en Wireshark que lo que viaja coincide con la especificación, que es justo lo que pide la Fase 2. También dejé decididos desde ahora la concurrencia, los errores, los temporizadores, el DNS, el LDAP y el log, para no tener que cambiar el protocolo cuando empiece a programar.

Queda un punto abierto: confirmar con el profesor el uso de `libldap` en el servidor. Esa biblioteca solo sirve para hablar con el directorio; los sockets del protocolo siguen construidos con la API Berkeley.

20. Referencias

Crocker, D. (Ed.), & Overell, P. (2008). *Augmented BNF for syntax specifications: ABNF* (RFC 5234). RFC Editor. https://www.rfc-editor.org/rfc/rfc5234

Eddy, W. (Ed.). (2022). *Transmission Control Protocol (TCP)* (RFC 9293). RFC Editor. https://www.rfc-editor.org/rfc/rfc9293

Kyzivat, P. (2014). *Case-sensitive string support in ABNF* (RFC 7405). RFC Editor. https://www.rfc-editor.org/rfc/rfc7405

Mockapetris, P. (1987a). *Domain names: Concepts and facilities* (RFC 1034). RFC Editor. https://www.rfc-editor.org/rfc/rfc1034

Mockapetris, P. (1987b). *Domain names: Implementation and specification* (RFC 1035). RFC Editor. https://www.rfc-editor.org/rfc/rfc1035

Postel, J. (1980). *User Datagram Protocol* (RFC 768). RFC Editor. https://www.rfc-editor.org/rfc/rfc768

Postel, J. (1981). *Transmission Control Protocol* (RFC 793). RFC Editor. https://www.rfc-editor.org/rfc/rfc793

Sermersheim, J. (Ed.). (2006). *Lightweight Directory Access Protocol (LDAP): The protocol* (RFC 4511). RFC Editor. https://www.rfc-editor.org/rfc/rfc4511

Universidad EAFIT. (2026a). *Capa de aplicación: DNS, HTTP, SMTP, LDAP y NFS* [Material de curso]. ST0255 Telemática: Internet, Arquitectura y Protocolos.

Universidad EAFIT. (2026b). *Capa de transporte* [Material de curso]. ST0255 Telemática: Internet, Arquitectura y Protocolos.

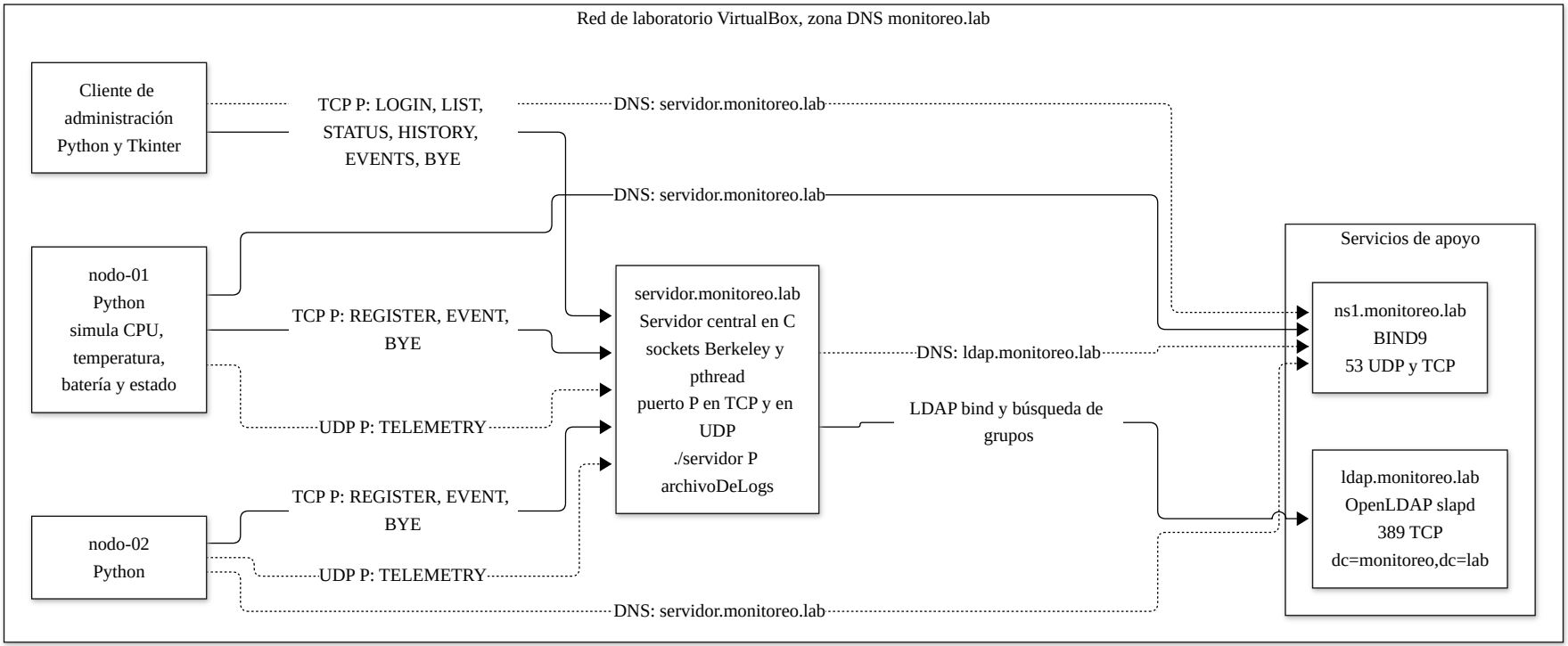
Universidad EAFIT. (2026c). *Enunciado del proyecto "Sistema de monitoreo y control distribuido"* [Material de curso]. ST0255 Telemática: Internet, Arquitectura y Protocolos.

Universidad EAFIT. (2026d). *Programación de sockets* [Material de curso]. ST0255 Telemática: Internet, Arquitectura y Protocolos.

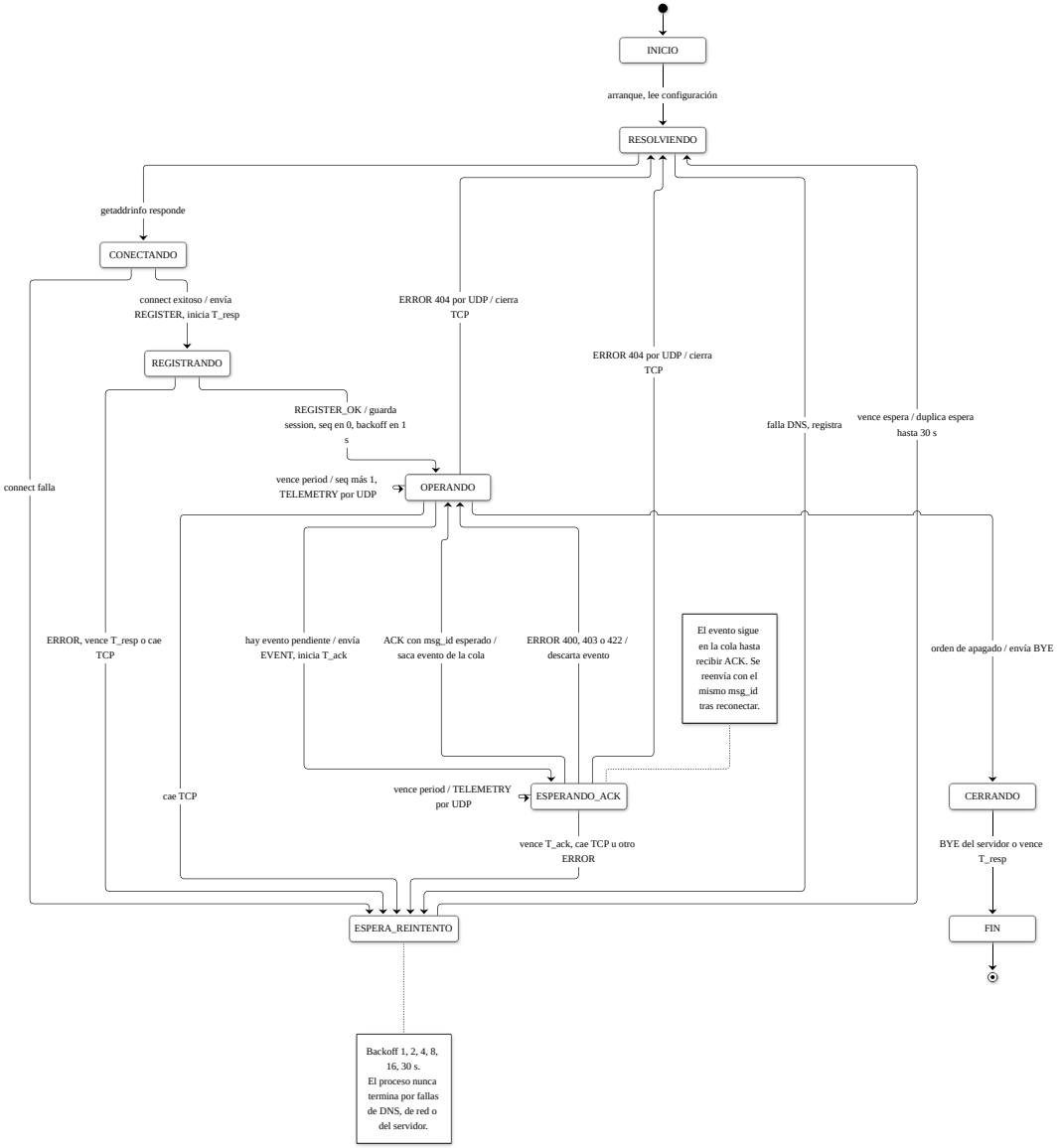
Anexo A. Diagramas

Los diagramas resumen la arquitectura, los estados y los intercambios principales. Las tablas de transición de docs/protocolo/MCD-1.0.md son la referencia normativa.

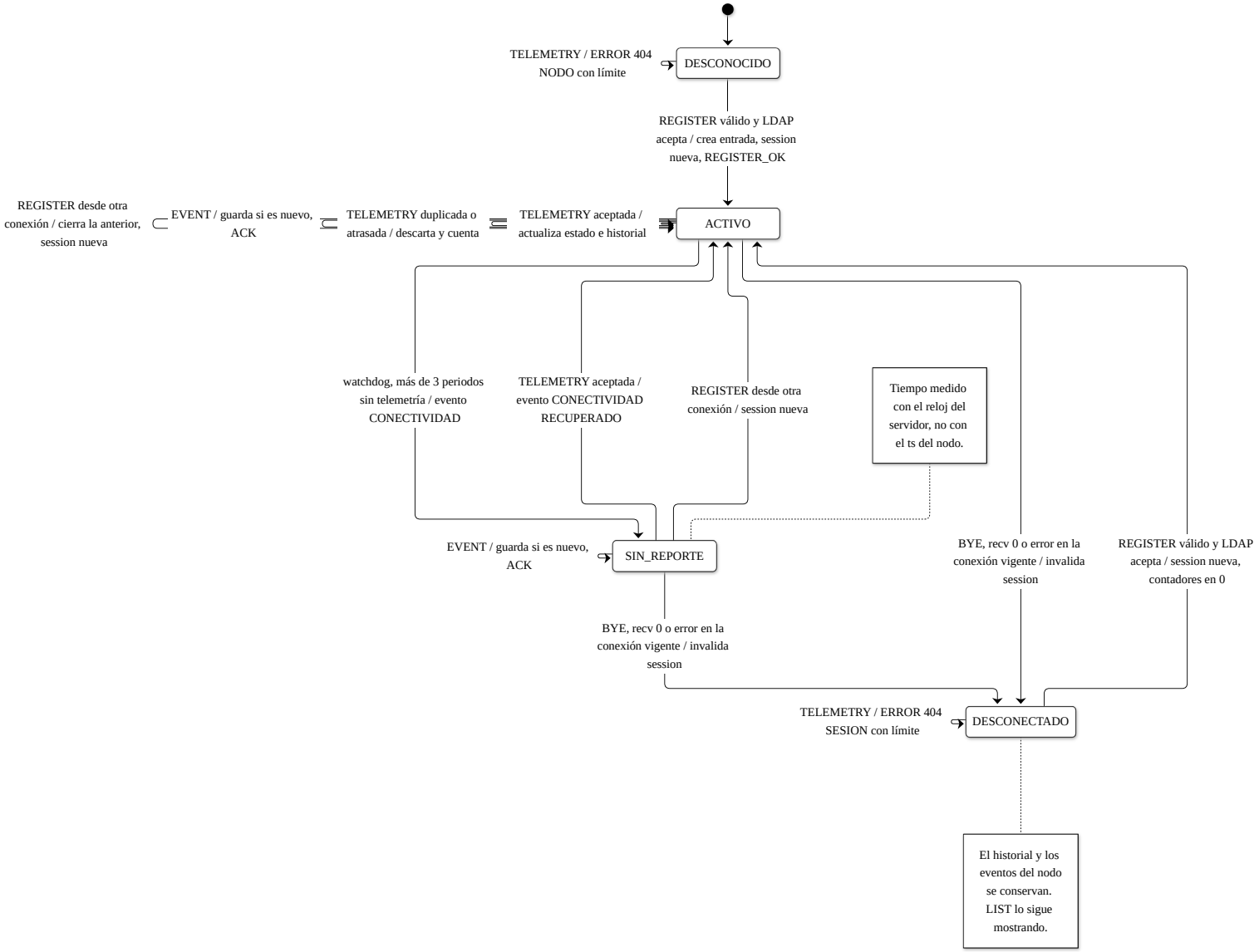
Arquitectura de despliegue



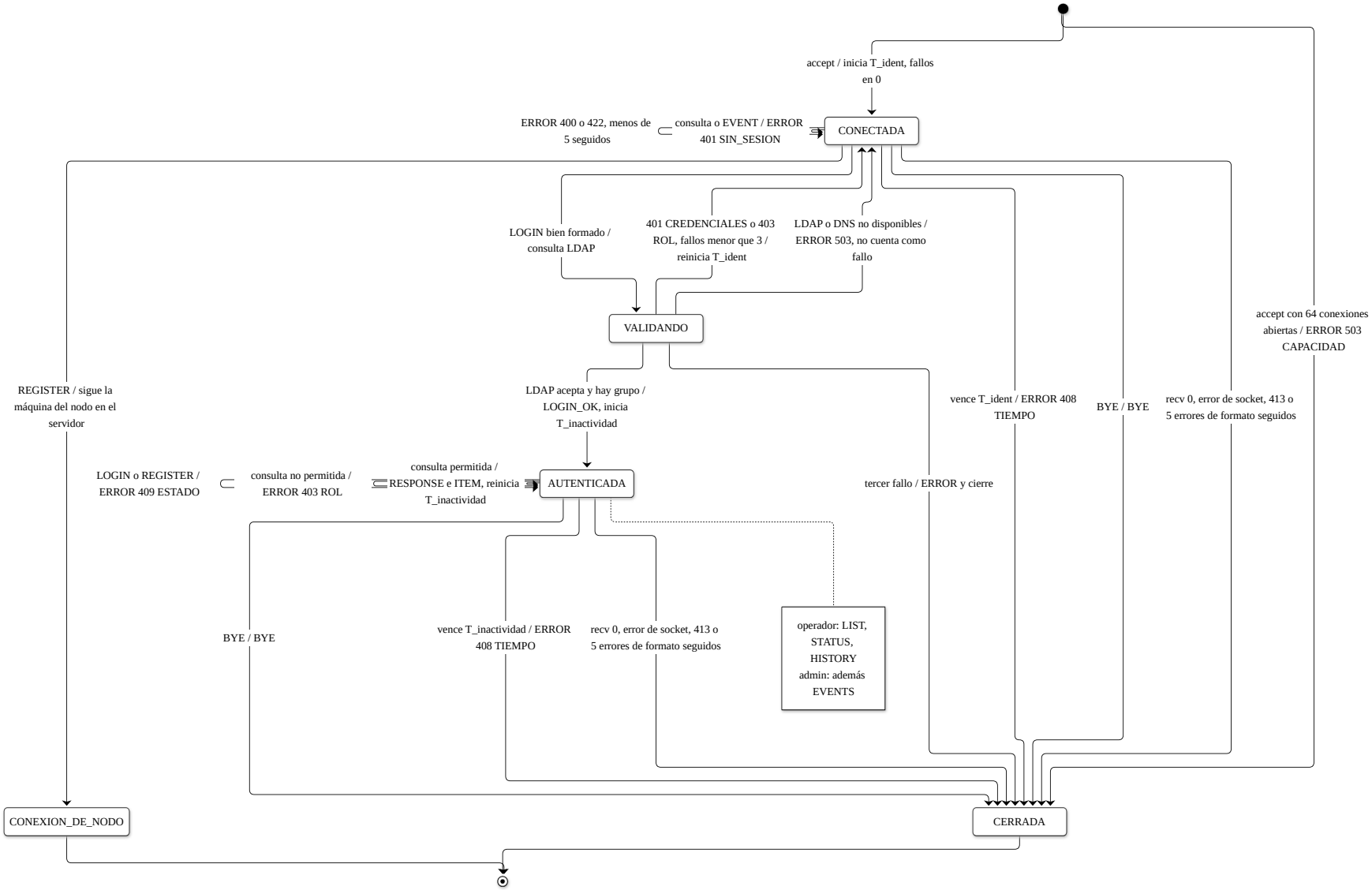
Máquina de estados del nodo



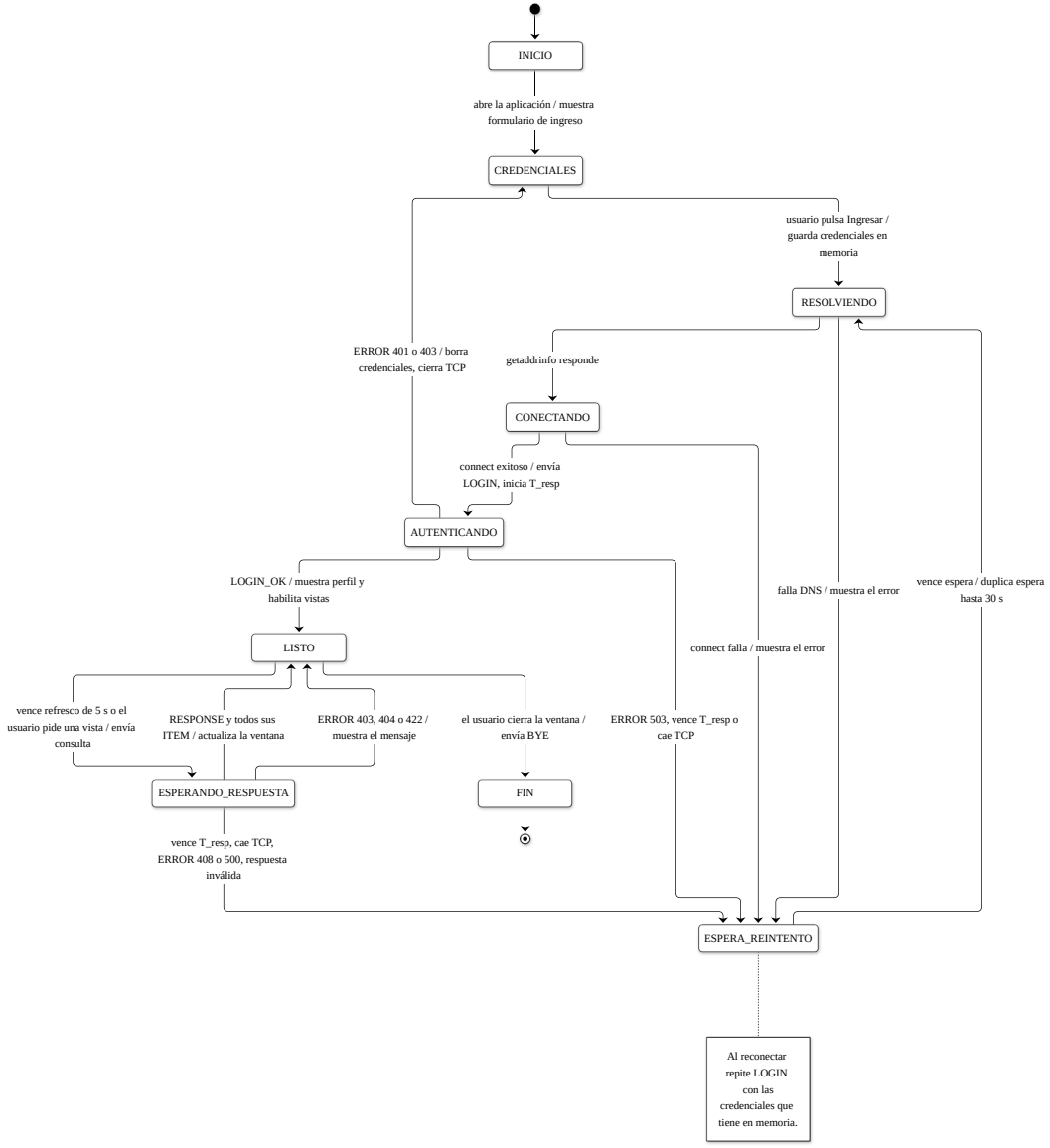
Estado del nodo visto por el servidor



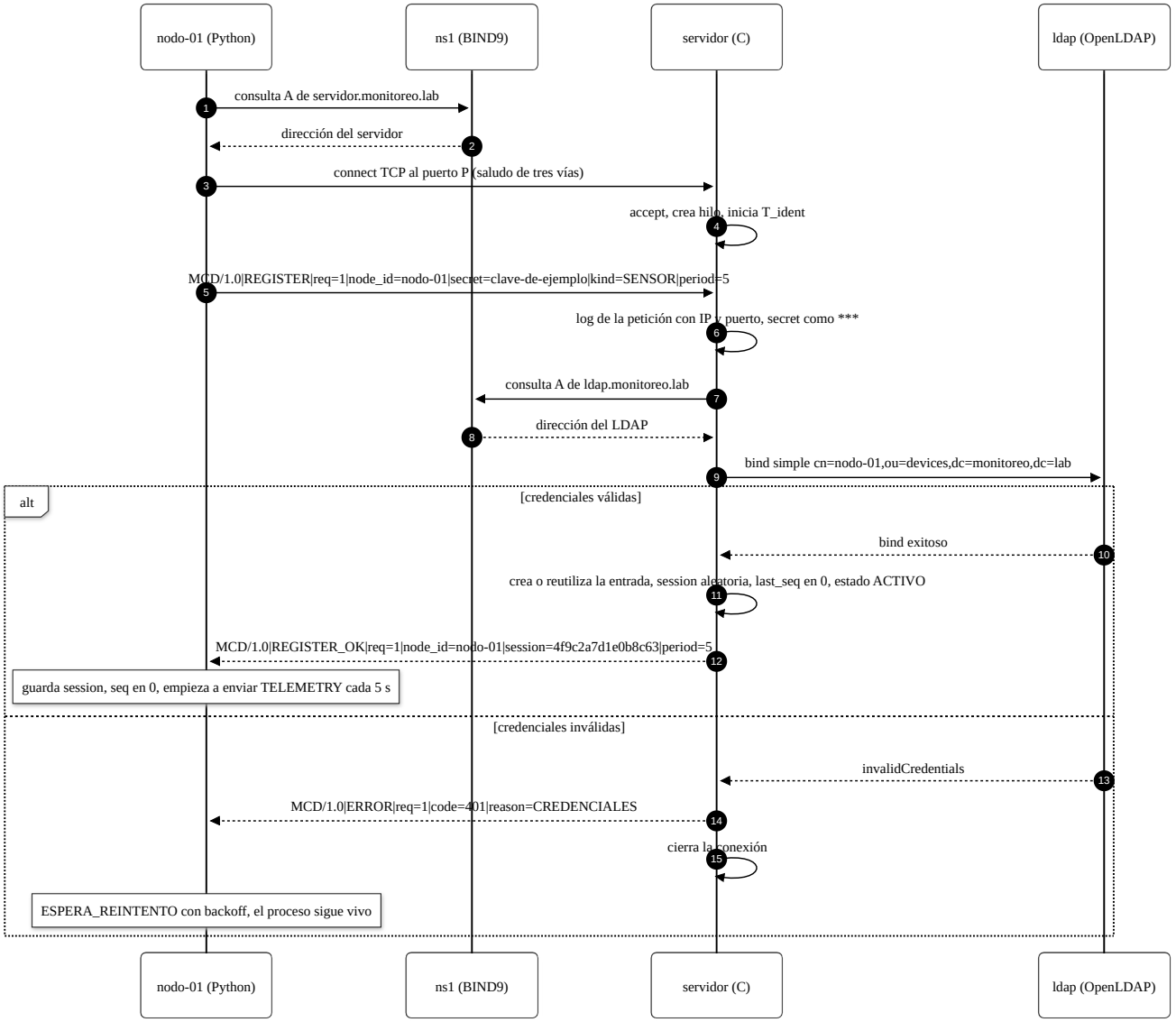
Sesión administrativa en el servidor



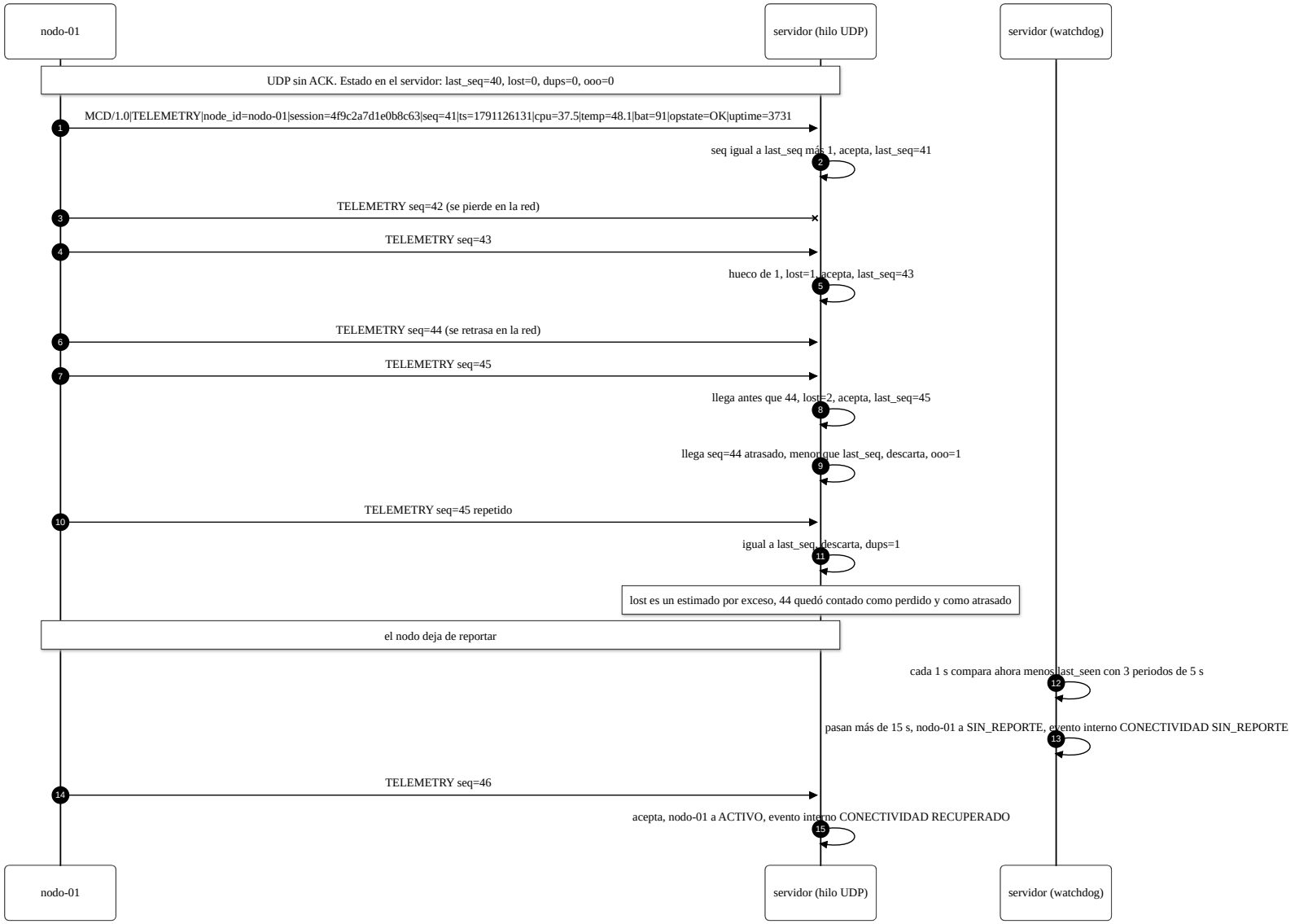
Máquina de estados del cliente administrativo



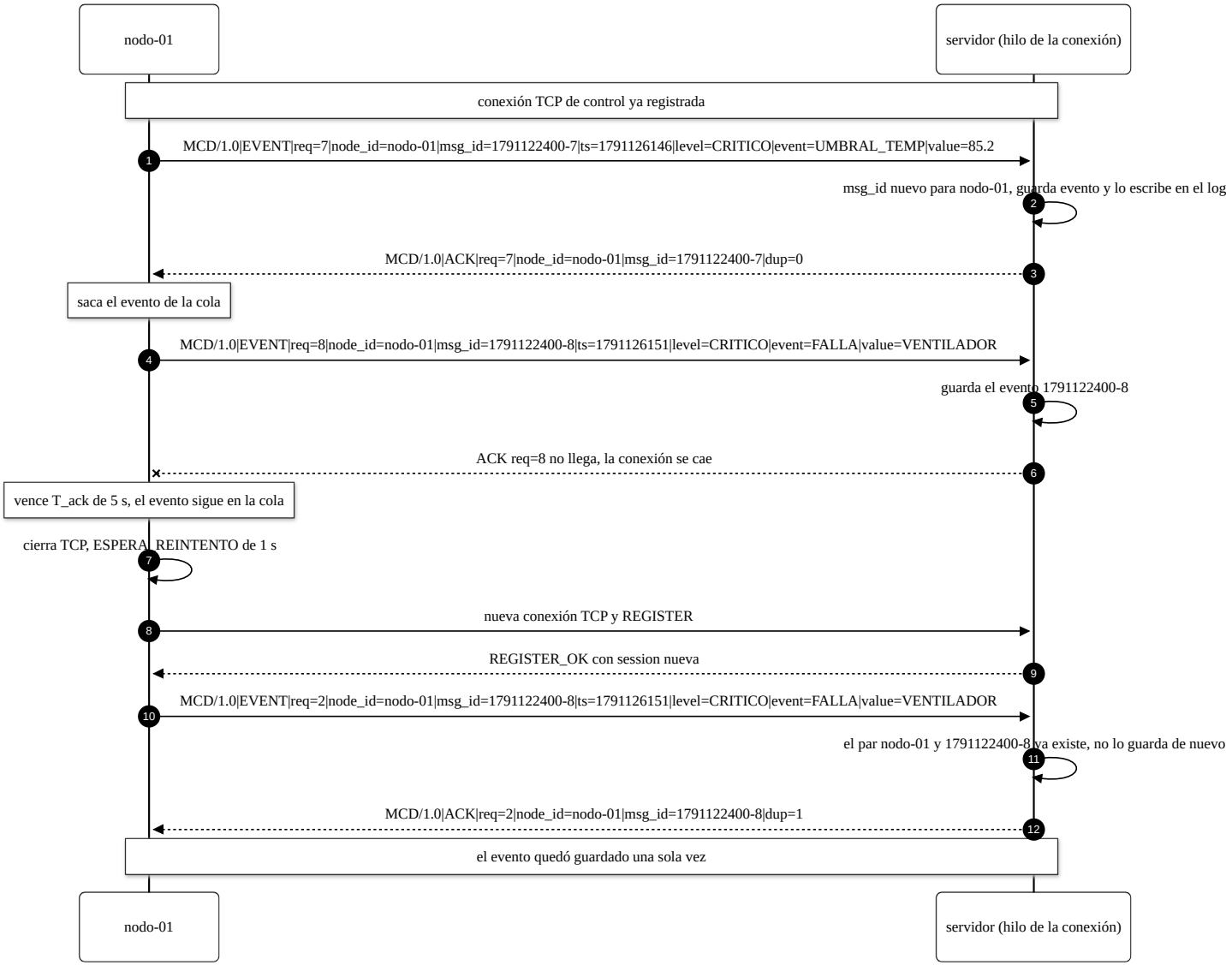
Secuencia de registro



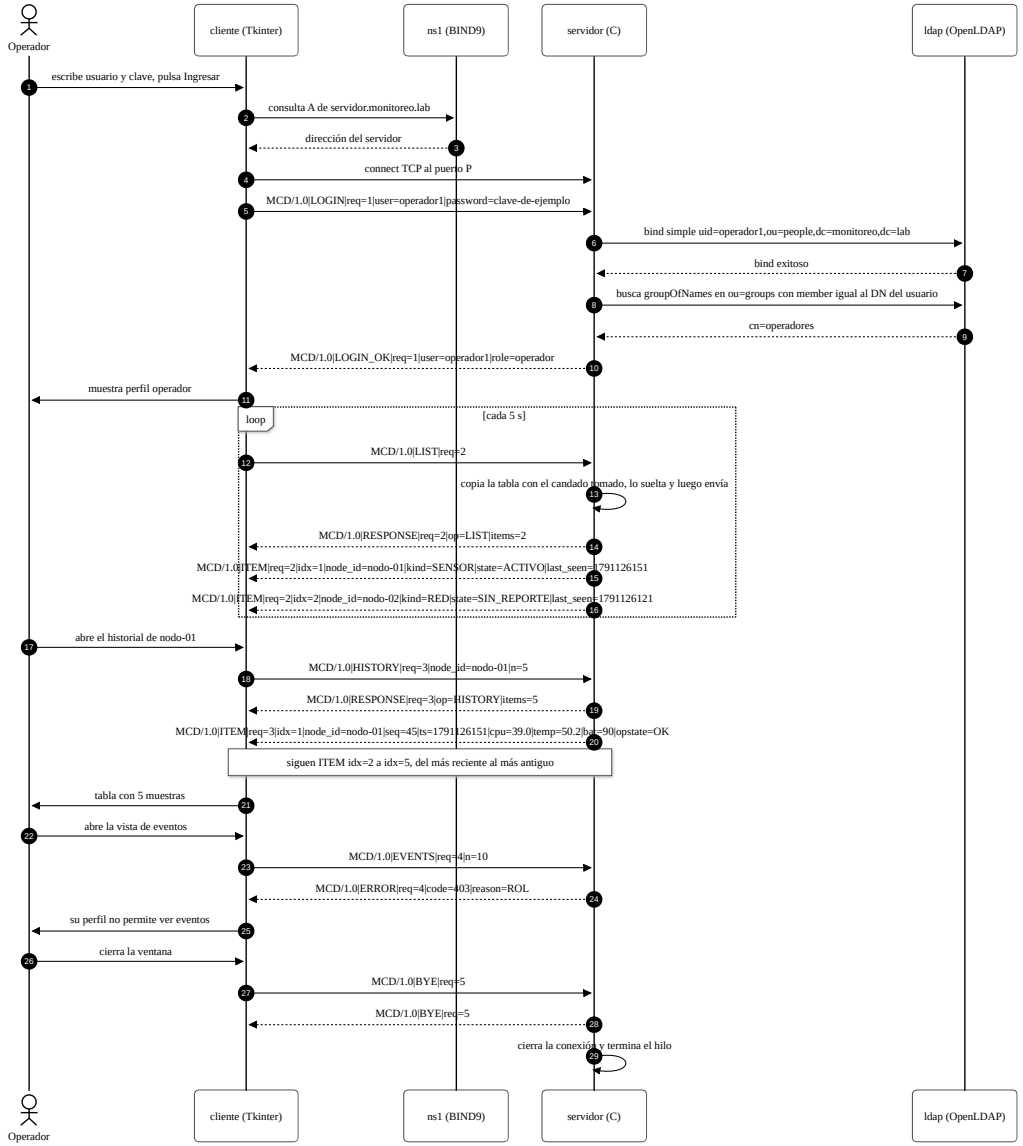
Secuencia de telemetría



Secuencia de evento crítico



Secuencia de autenticación y consultas



Errores del protocolo



Comportamiento ante servicios caídos

